

Evidencia de Contenerización y Despliegue

Semana 4 — Contenedores, despliegue y costos

Proyecto	NovaTareas Pro — Asistente inteligente de gestión de tareas con IA
Módulo	Módulo 4 — Desarrollo de Aplicaciones con IA
Equipo	Equipo 3
Integrantes	Saúl Oswaldo López Hernández — SMIS108421 Moises Antonio Martínez — SMIS071221 Enson Onan Carranza Rodríguez — SMIS013020
Docente	Ing. Marco Arévalo Zambrano
Repositorio	github.com/Saul1hdz/NovaTareas
Fecha	27 de julio de 2026

Resumen de la entrega

Elemento	Estado
Ruta elegida	Docker multi-stage + Docker Compose
Imagen de producción	Target runtime sobre Node 22.23.1 slim, usuario no root
Orquestación	compose.prod.yml: PostgreSQL 16, migraciones y web
Exclusiones de build	.dockerignore con 23 reglas
Variables de entorno	.env.example versionado, sin secretos reales
Sondas de salud	/api/v1/health y /api/v1/health/ready
Infraestructura	Servidor Netcup, Debian 13, 8 vCPU / 16 GB RAM
Costo estimado	USD 50.00 por la fase del proyecto

1. Descripción del servicio preparado

NovaTareas Pro es una aplicación web con bot de Telegram integrado que ayuda a organizar tareas usando inteligencia artificial. Lo que se contenerizó en esta entrega es el servicio completo: el servidor web con renderizado en el lado del servidor, la base de datos PostgreSQL 16 y el proceso de migraciones que prepara el esquema antes de que la aplicación arranque.

El servicio expone tres superficies de uso que comparten la misma base de datos y la misma lógica de negocio: el panel web, el bot de Telegram y una API REST protegida que permite consumir la capacidad de IA desde clientes externos.

Componentes contenerizados

Servicio	Imagen o target	Función
db	postgres:16-alpine	Motor de base de datos. Sin puertos publicados: solo accesible desde la red interna.
migrate	target runtime	Aplica las migraciones y termina. La web no arranca hasta que este proceso finaliza con éxito.
web	target runtime	Servidor Astro SSR. Escucha solo en loopback; el proxy inverso publica el servicio con TLS.
bot	target runtime	Bot de Telegram por polling. Está detrás de un perfil opcional para evitar dos instancias con el mismo token.

2. Ruta elegida: Docker

De las tres opciones planteadas —Docker, plataforma en la nube o simulación profesional— el equipo eligió **Docker con Compose**. Las razones son concretas y responden a problemas que ya habíamos tenido:

Diferencias de entorno entre integrantes. El proyecto exige Node 22.12 o superior. Durante el desarrollo aparecieron fallos causados precisamente por versiones distintas de Node entre las máquinas del equipo. El contenedor fija la versión y elimina esa clase de error.

La base de datos dejó de ser opcional. Tras migrar de SQLite a PostgreSQL 16, el motor pasó a ser un servicio aparte que hay que levantar, migrar y mantener sano. Compose resuelve ese orden de arranque de forma declarativa en lugar de depender de instrucciones manuales.

Continuidad hacia el despliegue real. El mismo Dockerfile sirve para desarrollo y para el servidor de producción, cambiando únicamente el target de construcción. Eso reduce la distancia entre lo que se prueba en local y lo que finalmente se ejecuta en el servidor.

3. Dockerfile

El Dockerfile usa construcción por etapas. La etapa `dependencies` instala los paquetes una sola vez y es reutilizada por las demás; `development` sirve para el trabajo diario con recarga en caliente; `build` compila el proyecto; y `runtime` arma la imagen final, que no incluye dependencias de desarrollo ni código fuente innecesario.

```
FROM node:22.23.1-bookworm-slim AS dependencies
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM dependencies AS development
ENV NODE_ENV=development
COPY . .
EXPOSE 4321
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]

FROM dependencies AS build
COPY . .
RUN npm run build

FROM node:22.23.1-bookworm-slim AS runtime
ENV NODE_ENV=production
# El adaptador de Astro lee HOST y PORT. Sin HOST=0.0.0.0 el proceso solo
# aceptaría conexiones desde dentro del propio contenedor.
ENV HOST=0.0.0.0
ENV PORT=4321
# tini como PID 1: reenvía SIGTERM al proceso de Node para que `docker stop`
# cierre limpiamente en lugar de esperar diez segundos y matarlo.
RUN apt-get update \
    && apt-get install -y --no-install-recommends tini \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/dist ./dist
# `migrations` y `scripts` permiten ejecutar db:pg:migrate y db:seed dentro del
# contenedor; `src` es su dependencia (esquema y capa de datos). No se copia
# drizzle.config.mjs: drizzle-kit es dependencia de desarrollo y no está aquí,
# así que generar migraciones es tarea de la máquina de desarrollo.
COPY --from=build /app/migrations ./migrations
COPY --from=build /app/scripts ./scripts
COPY --from=build /app/src ./src
# Los avatares se sirven desde dist/client. El directorio se crea aquí para
# poder montar encima un volumen persistente con el dueño correcto.
RUN mkdir -p dist/client/avatars && chown -R node:node /app
USER node
EXPOSE 4321
HEALTHCHECK --interval=15s --timeout=5s --start-period=20s --retries=3 \
    CMD node -e "fetch('http://127.0.0.1:$(process.env.PORT)|4321)+
    '/api/v1/health/ready').then(r=>process.exit(r.ok?0:1))
    .catch(()=>process.exit(1))"
ENTRYPOINT ["usr/bin/tini", "--"]
CMD ["node", "dist/server/entry.mjs"]
```

Figura 1. Archivo Dockerfile del repositorio.

Decisiones de seguridad y operación

Decisión	Motivo
USER node	El proceso no corre como root. Si alguien lograra ejecutar código dentro del contenedor, no tendría privilegios administrativos.
tini como PID 1	Node no maneja bien las señales cuando es el proceso inicial. Con tini, docker stop cierra limpio en lugar de esperar el tiempo de espera y matar el proceso.
HOST=0.0.0.0	Sin esta variable el adaptador de Astro solo acepta conexiones desde dentro del contenedor, y el servicio parecería caído desde fuera.
npm ci --omit=dev	La imagen final no incluye herramientas de desarrollo ni de pruebas: menos peso y menos superficie de ataque.
HEALTHCHECK	Docker comprueba cada 15 segundos que el servicio responde y que la base de datos está accesible, no solo que el proceso exista.

4. Exclusiones del contexto de construcción

El archivo `.dockerignore` define qué no se envía al motor de Docker al construir la imagen. Sin él, el contexto incluiría el historial de Git, las dependencias instaladas en la máquina local y el archivo `.env` con credenciales reales.

```
.git
.github
.astro
.env
.env.*
!.env.example
node_modules
dist
coverage
db
backups
tmp
output
*.log
npm-debug.log*
# Nada de esto interviene en el build ni en el
runtime: solo engorda el
# contexto de construccion y la imagen final.
tests
docs
data
.playwright-cli
*.md
!README.md
```

Figura 2. Archivo `.dockerignore` del repositorio.

Razón de cada grupo de exclusiones

Exclusión	Por qué se excluye
<code>.env, .env.*</code>	Seguridad. Impide que las credenciales reales queden dentro de la imagen. La excepción <code>!.env.example</code> conserva la plantilla, que solo tiene marcadores de posición.
<code>node_modules</code>	Las dependencias se instalan dentro del contenedor con <code>npm ci</code> . Copiar las de la máquina local rompería los módulos compilados para otro sistema operativo.
<code>.git, .github</code>	El historial de versiones y los flujos de integración continua no se usan en tiempo de ejecución y pueden pesar bastante.
<code>dist, .astro</code>	Artefactos de compilación previos. La imagen los genera desde cero para garantizar que corresponden al código copiado.
<code>tests, coverage</code>	Las pruebas se ejecutan en integración continua, no dentro de la imagen de producción.
<code>docs, *.md</code>	Documentación que no interviene en la ejecución. Se conserva <code>README.md</code> como referencia mínima.
<code>db, backups, output</code>	Datos locales y respaldos. Nunca deben viajar dentro de una imagen que puede publicarse.

5. Orquestación de producción

El archivo `compose.prod.yml` define cómo se levanta el servicio en el servidor. Se muestra un extracto con las decisiones más relevantes.

```
name: novatareas-prod

x-app-environment: &app-environment
  NODE_ENV: production
  DATABASE_URL: postgresql://novatareas:${POSTGRES_PASSWORD:?define POSTGRES_PASSWORD}@db:5432/novatareas
  SECRET_KEY: ${SECRET_KEY:?define SECRET_KEY}
  CRON_SECRET: ${CRON_SECRET:?define CRON_SECRET}
  APP_TIME_ZONE: ${APP_TIME_ZONE:-America/EL_Salvador}

services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    # Sin `ports`: la base solo es accesible desde la red interna del compose.
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U novatareas -d novatareas"]
    volumes:
      - novatareas_prod_pgdata:/var/lib/postgresql/data

  migrate:
    build: { context: ., target: runtime }
    restart: "no"
    depends_on:
      db: { condition: service_healthy }
    command: ["node", "scripts/migrate-postgres.mjs"]

  web:
    build: { context: ., target: runtime }
    restart: unless-stopped
    depends_on:
      migrate: { condition: service_completed_successfully }
    # Solo escucha en loopback: el proxy inverso con TLS publica el servicio.
    ports:
      - "127.0.0.1:${WEB_PORT:-4321}:4321"
    volumes:
      - novatareas_prod_avatars:/app/dist/client/avatars
```

Figura 3. Extracto de `compose.prod.yml`.

Diferencias frente al entorno de desarrollo

Aspecto	Desarrollo	Producción
Target de la imagen	development	runtime
Código fuente	Montado como volumen	Copiado y compilado en la imagen
Puerto de PostgreSQL	Publicado en 5434	No se publica
Puerto de la web	Abierto en la máquina	Solo loopback, detrás del proxy
Secretos	Valores por defecto para pruebas	Obligatorios; el arranque falla si faltan
Sonda de salud	Comprueba el proceso	Consulta la base de datos

El operador `:?` en las variables obliga a definirlos: si `POSTGRES_PASSWORD` o `SECRET_KEY` faltan, Compose se niega a arrancar en lugar de levantar el servicio con valores inseguros.

6. Variables de entorno

El repositorio versiona `.env.example` con marcadores de posición. Los archivos `.env` y `.env.local` están excluidos por `.gitignore` y por `.dockerignore`, de modo que no llegan al repositorio ni a la imagen.

Obligatorias

Variable	Descripción	Consecuencia si falta
DATABASE_URL	Cadena de conexión a PostgreSQL 16, único motor soportado.	La aplicación no arranca.
SECRET_KEY	Secreto para firmar las sesiones.	Compose se niega a iniciar en producción.
POSTGRES_PASSWORD	Contraseña del motor en el servidor.	Compose se niega a iniciar.
ZAI_API_KEY	Credencial del proveedor de IA.	El sistema recurre a la cascada de respaldo.

Opcionales, según la función que se pruebe

Variable	Para qué sirve
TEST_DATABASE_URL	Base de pruebas. El nombre debe terminar en <code>_test</code> : el arranque de la suite borra su esquema completo y se niega a apuntar a otra cosa.
TELEGRAM_BOT_TOKEN	Token del bot. Solo necesario si se va a probar Telegram.
AI_API_KEY	Clave propia para consumir la API externa de recomendaciones. No se reutiliza la del proveedor.
CRON_SECRET	Protege el endpoint de recordatorios programados.
TOKEN_ENCRYPTION_KEY	Cifra los tokens de Google Calendar. Debe ser un valor de 32 bytes en base64url.
APP_TIME_ZONE	Zona horaria del servicio. Por defecto <code>America/El_Salvador</code> .
PG_POOL_MAX	Tamaño máximo del grupo de conexiones a PostgreSQL. Por defecto 10.

Nota de seguridad. En una entrega anterior se detectó que `.env.example` contenía credenciales reales en lugar de marcadores. Las claves afectadas fueron rotadas y el archivo reescrito. Ese hallazgo está documentado en el registro de la Semana 3.

7. Evidencia de construcción y ejecución

7.1. Construcción de la imagen

```
$ docker compose -f compose.prod.yml build

[+] Building ... FINISHED
=> [dependencies 1/4] FROM docker.io/library/node:22.23.1-bookworm-slim
=> [dependencies 4/4] RUN npm ci
=> [build 2/2] RUN npm run build
=> [runtime 4/9] RUN npm ci --omit=dev && npm cache clean --force
=> [runtime 9/9] RUN mkdir -p dist/client/avatars && chown -R node:node /app
=> exporting to image
```

Figura 4. Construcción de la imagen de producción por etapas.

7.2. Levantamiento del servicio

```
$ docker compose -f compose.prod.yml up -d

✓ Container novatareas-prod-db-1 Healthy
✓ Container novatareas-prod-migrate-1 Exited (0)
✓ Container novatareas-prod-web-1 Started

$ docker compose -f compose.prod.yml ps
NAME STATUS
novatareas-prod-db-1 Up (healthy)
novatareas-prod-web-1 Up (healthy)
```

Figura 5. Orden de arranque: la base queda sana, las migraciones terminan con código 0 y solo entonces inicia la web.

El estado `healthy` del contenedor web no lo declara el operador: proviene de la instrucción `HEALTHCHECK` del `Dockerfile`, que consulta `/api/v1/health/ready` cada 15 segundos. Esa sonda comprueba también que la base de datos responde, así que un contenedor sano implica que la cadena completa funciona.

8. Prueba del endpoint de salud

```
$ curl -s http://127.0.0.1:4321/api/v1/health

{
  "status": "ok",
  "service": "novatareas-ai",
  "timestamp": "2026-07-27T14:20:11.482Z",
  "checks": {
    "zai_configured": true,
    "ollama_available": false,
    "fallback_rules": true
  }
}
```

Figura 6. Respuesta de GET /api/v1/health (código 200).

El servicio informa además el estado de sus dependencias. El campo `fallback_rules` siempre es verdadero: aunque el proveedor de IA y el modelo local no estén disponibles, el sistema puede responder con reglas locales.

9. Prueba del endpoint principal

El endpoint que expone la capacidad inteligente del proyecto es `POST /api/v1/recommend`. Requiere autenticación mediante una clave propia, distinta de la credencial del proveedor de IA.

9.1. Petición válida

```
$ curl -s -X POST http://127.0.0.1:4321/api/v1/recommend \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $AI_API_KEY" \
-d '{"titulo":"Estudiar para el examen de calculo",
    "prioridad":"alta","tipo_usuario":"estudiante"}'

{
  "recomendacion": "Divide el temario en tres bloques antes de empezar. ...",
  "fuente": "zai",
  "tarea": { "titulo": "Estudiar para el examen de calculo",
    "prioridad": "alta",
    "tipo_usuario": "estudiante" }
}
```

Figura 7. Respuesta correcta (código 200). El campo `fuentes` indica qué motor generó la recomendación.

9.2. Error controlado

```
$ curl -s -i -X POST http://127.0.0.1:4321/api/v1/recommend \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $AI_API_KEY" \
-d '{"prioridad":"alta"}'

HTTP/1.1 400 Bad Request
content-type: application/json

{ "error": "El campo \"titulo\" es obligatorio y no puede estar vacio." }
```

Figura 8. Entrada inválida rechazada con un mensaje comprensible, sin exponer detalles internos.

10. Errores, bloqueos y correcciones

Los problemas siguientes surgieron al preparar la contenerización y el despliegue. Se documentan tal como ocurrieron, incluidos los que obligaron a rehacer decisiones ya tomadas.

10.1. Retirada completa de SQLite

Bloqueo. El proyecto usaba SQLite mediante un módulo compilado de forma nativa. En un contenedor eso obliga a instalar herramientas de compilación en la imagen, y el archivo de base de datos vive dentro del sistema de archivos del contenedor, que es efímero: al recrearlo se pierden los datos. Además SQLite no admite escrituras concurrentes, lo que impide escalar la aplicación.

Corrección. Se migró a PostgreSQL 16 como motor único. Se retiraron el controlador, las migraciones antiguas, el importador y la capa de compatibilidad. Las pruebas se reescribieron para ejecutarse contra PostgreSQL real, el mismo motor que se despliega, y una prueba automatizada verifica que nadie reintroduzca dialecto de SQLite en el código.

10.2. La bandera de Node dejó de existir

```
$ npm run bot:dev  
> node --experimental-sqlite telegram/bot.js  
node: bad option: --experimental-sqlite
```

Diagnóstico. Los scripts arrastraban una bandera experimental que las versiones nuevas de Node ya no aceptan. El error confirmó que fijar la versión del entorno no era opcional. **Corrección.** Se eliminó la bandera y se fijó la versión mediante `.nvmrc`, el campo `engines` y la imagen base del Dockerfile, que ahora es la misma para todo el equipo.

10.3. Finales de línea de Windows dentro del contenedor

Bloqueo. El equipo trabaja en Windows, donde Git puede convertir los finales de línea al formato CRLF. Los archivos que se ejecutan dentro de un contenedor Linux fallan con ese formato. **Corrección.** Se agregó `.gitattributes` forzando finales LF, y se documentaron los tropiezos propios de Docker Desktop en Windows en un archivo aparte para el resto del equipo.

10.4. Riesgo de pérdida de datos al detener el servicio

Bloqueo. Detener el entorno con la opción que elimina volúmenes borra la base de datos, las dependencias del contenedor y los avatares subidos. Es un comando de uso frecuente y la diferencia con la versión segura es un solo carácter. **Corrección.** Los datos se aislaron en volúmenes con nombre y se documentó de forma explícita que no debe usarse esa opción, además de dejar escrito el procedimiento de restauración a partir de un respaldo.

10.5. Una sola instancia del bot por token

Bloqueo. El bot funciona por sondeo. Si dos personas levantan el contenedor con el mismo token, las instancias se roban los mensajes entre sí. **Corrección.** El bot quedó detrás de un perfil de Compose que no arranca por defecto: hay que pedirlo de forma explícita. El estado de la conversación se movió a PostgreSQL, de modo que sobrevive a los reinicios del contenedor.

11. Plan de infraestructura mínima

El servicio se desplegará en un servidor propio contratado para la fase del proyecto. La topología prevista es la siguiente:

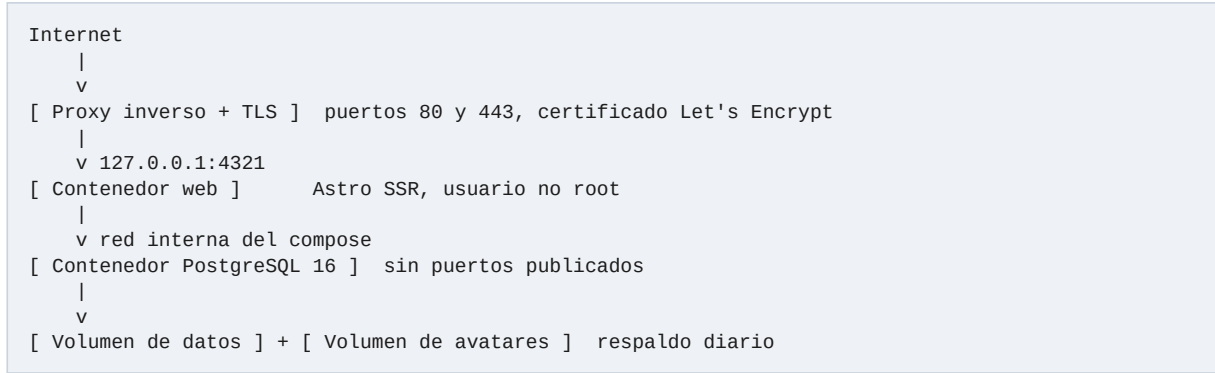


Figura 9. Topología prevista en el servidor.

Especificaciones del servidor contratado

Característica	Valor
Proveedor	Netcup
Ubicación	Manassas, Virginia, Estados Unidos
Tipo	Root Server virtualizado con KVM
CPU	8 vCPU — AMD EPYC 9645
Memoria	16 GB RAM
Almacenamiento	512 GB virtual / 503 GiB utilizables, sistema de archivos ext4
Arquitectura	x86_64
Sistema operativo	Debian GNU/Linux 13 (Trixie), kernel Linux 6.12.95

El dimensionamiento es holgado para el uso esperado. La aplicación es un monolito modular con una sola instancia de web y una de bot, y la carga prevista durante el módulo es de unos pocos usuarios simultáneos. El margen disponible permite alojar en la misma máquina el proxy inverso, los respaldos y, si más adelante se decide, una instancia local de Ollama para el respaldo de IA sin costo adicional por consumo.

Servicios mínimos a configurar

Componente	Función
Docker Engine y Compose	Ejecutar los contenedores definidos en <code>compose.prod.yml</code> .
Proxy inverso	Terminar TLS y publicar el servicio, que solo escucha en loopback.
Certificado TLS	Emisión y renovación automática con Let's Encrypt, sin costo.
Cortafuegos	Abrir únicamente los puertos 22, 80 y 443.
Respaldo programado	Volcado diario de PostgreSQL y del volumen de avatares.
Dominio	Registro propio para el acceso público con HTTPS.

12. Estimación de costos iniciales

Durante la fase del proyecto el equipo mantiene rentado un servidor con el proveedor Netcup, con un costo de **USD 50.00**. El resto de los componentes se cubre con planes gratuitos o con costos marginales.

Concepto	Proveedor	Costo (USD)	Observación
Servidor de la fase del proyecto	Netcup	50.00	Root Server KVM, 8 vCPU, 16 GB RAM, 512 GB. Incluye tráfico y dirección IP.
Dominio	Registrador externo	12.00	Estimado anual para un dominio genérico. Prorrateado a la fase: menos de 3 USD.
Certificado TLS	Let's Encrypt	0.00	Emisión y renovación automáticas.
Modelo de IA	z.ai	0.00	Modelo g1m-4.5-flash dentro del plan gratuito.
Bot de mensajería	Telegram	0.00	La API de bots no tiene costo.
Integración continua	GitHub Actions	0.00	Sin costo para repositorios públicos.
Base de datos	Autoalojada	0.00	PostgreSQL 16 en contenedor, dentro del mismo servidor.
Total de la fase		~62.00	Servidor más dominio anual completo.

Supuestos de la estimación

Esta estimación se sostiene sobre supuestos explícitos, que conviene revisar si cambian las condiciones del proyecto:

Supuesto	Detalle
Duración	El monto de USD 50.00 cubre el servidor durante la fase del proyecto, no es una tarifa mensual recurrente después de esa fase.
Volumen de uso	Pocos usuarios simultáneos, propios del equipo y de las demostraciones. No se contempla tráfico abierto al público.
Consumo de IA	Se permanece dentro del plan gratuito del proveedor. Si la cuota se agota, el sistema recurre al respaldo local sin costo, aunque con menor calidad de respuesta.
Almacenamiento	Los 512 GB son suficientes con margen amplio: la base de datos y los avatares del período de evaluación no superan unos pocos gigabytes.
Sin alta disponibilidad	Una sola máquina, sin réplica ni balanceo. Un fallo del servidor implica interrupción del servicio hasta restaurarlo.
Respaldos locales	Los volcados se guardan en el mismo servidor. Externalizarlos añadiría costo de almacenamiento, aún no contemplado.
Mano de obra	No se contabiliza el tiempo del equipo, al tratarse de un proyecto académico.

Proyección si el servicio siguiera activo. Manteniendo el mismo servidor y agregando la renovación anual del dominio, el costo recurrente se mantendría en el mismo orden de magnitud. El principal riesgo de aumento no es la infraestructura sino el consumo de IA: si el uso creciera lo suficiente para superar el plan gratuito, habría que contratar cuota de pago o desplazar la carga al modelo local, que ya está contemplado en la arquitectura.

13. Riesgos técnicos pendientes

Se documentan sin omitir los que siguen abiertos al cierre de esta entrega.

Riesgo	Descripción	Impacto	Mitigación prevista
Despliegue aún no ejecutado	El servicio está preparado y probado en local, pero todavía no corre en el servidor con dominio y TLS.	Alto	Ejecutar el procedimiento ya documentado en el runbook del proyecto.
Punto único de fallo	Una sola máquina, una instancia de web y una de bot. No hay réplica ni balanceo.	Alto	Asumido para la fase académica. Escalar exigiría separar el bot y publicar varias instancias de web.
Respaldos sin verificar	El procedimiento de volcado está escrito, pero no se ha probado una restauración completa.	Alto	Ensayar una restauración sobre un entorno limpio antes de considerar el servicio estable.
Sin registro estructurado	Las llamadas al modelo no registran modelo usado, tokens consumidos ni latencia.	Medio	Incorporar registro estructurado y métricas básicas de operación.
Sin métrica de calidad	No existe forma de medir si las recomendaciones resultan útiles para el usuario.	Medio	Añadir un mecanismo de retroalimentación en las recomendaciones.
Recuperación de acceso	Se basa en preguntas de seguridad. Es correcta para el ámbito académico, pero insuficiente para un servicio público.	Medio	Sustituir por enlaces enviados a un canal verificado antes de cualquier apertura al público.
Cobertura de pruebas incompleta	Quedan sin cubrir a fondo la conversación del bot, el motor de recuperación semántica y el flujo visual de Google.	Medio	Ampliar la suite en las siguientes iteraciones.
Dependencia de un proveedor de IA	El agotamiento de la cuota degrada la calidad de las respuestas.	Bajo	La cascada de respaldo ya evita la interrupción del servicio.
Dashboard monolítico	Un solo archivo supera las 2.800 líneas, lo que dificulta el mantenimiento.	Bajo	Continuar la extracción por capas ya iniciada.

14. Conclusión

El servicio quedó contenerizado de extremo a extremo: imagen de producción por etapas con usuario sin privilegios, orquestación que garantiza el orden de arranque, secretos obligatorios, sondas de salud que consultan la base de datos y volúmenes que preservan los datos entre reinicios. La infraestructura está contratada y dimensionada, y el procedimiento de despliegue está escrito.

Lo que falta es ejecutar ese despliegue en el servidor y verificar el procedimiento de restauración. Ambos puntos quedan registrados como riesgos abiertos y constituyen el trabajo inmediato de la siguiente etapa.